Enumerative Program Synthesis with Observational Equivalence Reduction

Varun Varma

April 29th, 2020

Changes

- Works cited page sources have been changed to reflect the new research.
- Pseudocode for the synthesis engine has been written.
- The graph for displaying synthesis time is now represented on a logarithmic scale, so all points of data are visible.
- Data analysis has been updated to use percentage difference data and address the outlier.
- A diagram visualizing program generation in bottom-up enumerative synthesis has been added.

Future Work

- Write an abstract.
- Flesh out and add in-text citations to background.
- Write a conclusion.
- Format code blocks.

Background

Program synthesis is the problem of generating a program from a given specification. Enumerative program synthesis is a family of program synthesis algorithms where the search space is enumerated until a program is found. Bottom-up search is another program synthesis algorithm that generates programs from previously generated programs. Enumerative bottom-up program synthesis is a powerful program synthesis technique that produces complete programs on every iteration and allows for a variety of optimizations via heuristics. However, an issue with this technique is that it generates programs that produce the same output. For example, x and $(x * 2)/2$ are equivalent programs, yet both these programs will have to be added to the program bank for future use, leading to exponential efficiency loss. These equivalent programs can be optimized away by comparing the observed output of a program with all other generated programs and keeping only the smallest one. This technique, called observational equivalence reduction, has the potential to radically speed up enumerative bottom-up program synthesis. This project implements a bottom-up enumerative program synthesis engine with optional observational equivalence reduction and evaluates it on a benchmark of synthesis problems o test the effectiveness of observational equivalence reduction.

Materials and Methods

An enumerative program synthesis engine that could optionally apply observational equivalence reduction was benchmarked on nine bit-manipulation problems adapted from the *Hacker's Delight* book. Each problem was defined by 256 input-out pairs calculated from the desired function. The synthesizer inputs a specification of type $[(i8, i8); 256]$ and outputs the total number of programs iterated and a function of type $i8 \rightarrow i8$ that satisfies the specification. The synthesizer was evaluated on each problem in the benchmark, once with observational equivalence reduction and once without. The total synthesis time, size of synthesized program, and number of programs iterated were recorded into a .csv file. All benchmarks were run on an Intel Core i7-9750H and 32 GB of RAM on a single thread.

```
def synthesize(examples):
    bank = []
    while True:
        for program in bank:
            for operation in Operation.enumerate():
                new_program = operation(program)
                correct = True
                for (input, output) in examples:
                    if program(input) != output:
                        correct = False
                        break
                if correct:
                    return new_program
                else:
                    bank.append(new_program)
```

```
def synthesize_with_observational_equivalence_reduction(examples):
    bank = []
    map = {}
    while True:
        for program in bank:
            for operation in Operation.enumerate():
                new_program = operation(program)
                correct = True
                outputs = []
                for (input, output) in examples:
                    program_output = program(input)
                    outputs.append(program(input))
                    if program_output != output:
                        correct = False
                if correct:
                    return new_program
                else:
                    banked_program = map[outputs]
                    if banked_program != None:
                        if new_program.size() < banked_program.size():
                            map[outputs] == new_program
                            bank[banked_program] = new_program
                    else:
                        bank.append(new_program)
```

Figure 1. Pseudocode for the implementation of the synthesis engine with and without observational equivalence reduction.

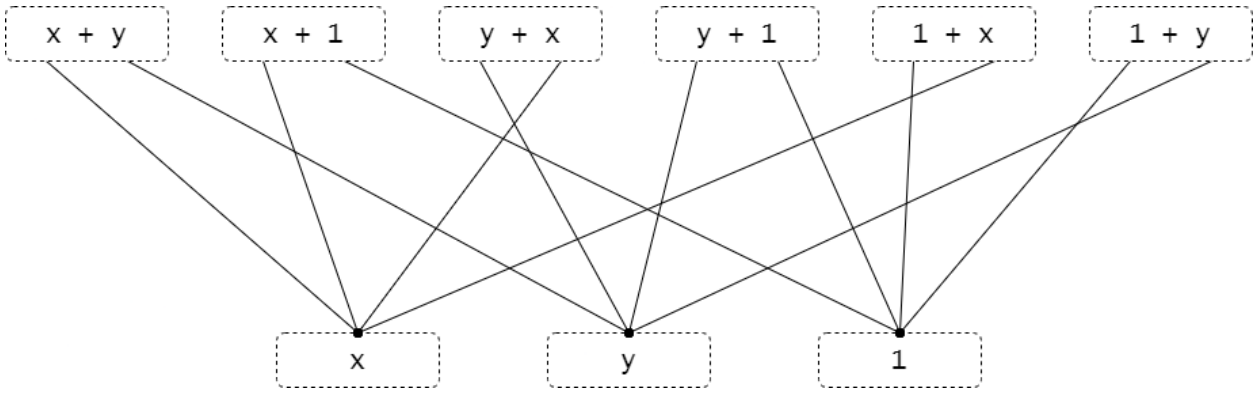

Figure 2. A diagram showing generation of height-2 programs from height-1 programs in bottom-up enumerative program synthesis.

Data

The enumerative program synthesis engine has been rewritten completely from Python to Rust, which will provide more speed and reliable data. The nine benchmark problems have been respecified to fit the new synthesis engine. The engine supports multi-argument specifications, so more problems from the *Hacker's Delight* problem set can be evaluated if necessary. The engine also features basic heuristics for traversing the search space, which allows it to synthesize up to height-3 programs, despite the search space of height-3 programs being millions of programs large.

```
// P1 ~ Turn off rightmost 1 bit
fn problem1(x: Value) -> Value {
    x & (x - Wrapping(1))
}

// P2 ~ Test whether an unsigned integer is of form 2^(n-1)
fn problem2(x: Value) -> Value {
    x & (x + Wrapping(1))
}

// P3 ~ Isolate rightmost bit
fn problem3(x: Value) -> Value {
    x & -x
}

// P4 ~ Form a mask that identifies the rightmost 1 bit and trailing 0s
fn problem4(x: Value) -> Value {
    x ^ (x - Wrapping(1))
}

// P5 ~ Right propagate rightmost 1 bit
fn problem5(x: Value) -> Value {
    x | (x - Wrapping(1))
}

// / P6 ~ Turn on rightmost 0 bit
```

```
fn problem6(x: Value) -> Value {
    x | (x + Wrapping(1))
}

// P7 ~ Isolate rightmost 0 bit
fn problem7(x: Value) -> Value {
    (!x) & (x + Wrapping(1))
}

// P8 ~ Form a mask that identifies trailing 0s
fn problem8(x: Value) -> Value {
    (x - Wrapping(1)) & (!x)
}

// P13 ~ Sign function
fn problem13(x: Value) -> Value {
    let o1 = x.rotate_right(MAX as u32);
    let o2 = -x;
    let o3 = o2.rotate_right(MAX as u32);
    return o1 | o3;
}
```

Figure 3: The nine problems from the *Hacker's Delight* problem set.

Problem	Synthesized Program Without Observational Equivalence Reduction	Synthesized Program With Observational Equivalence Reduction
1	(& (- x 1) (+ x 0))	(& (- x 1) x)
2	(& (^ x 0) (+ 1 x))	(& x (+ x 1))
3	(& (/ x 0) (- 0 x))	(& x (- 0 x))
4	(^ (- 0 x) (! x))	(^ (+ x -128) (+ 127 x))
5	(| (| 1 x) (- x 1))	(| (| x 1) (- x 1))
6	(| (| 1 x) (+ 1 x))	(| (| x 1) (+ x 1))
7	(& (+ x 1) (! x))	(& (+ 1 x) (! x))
8	(& (- x 1) (! x))	(& (! x) (- x 1))
9	(& (^ (- 0 x) (^ 1 x)) (/ (- 0 x) (>> x 0)))	(& (^ (- 0 x) (^ x 1)) (/ (- 0 x) x))

Figure 4: A table of synthesized programs for nine problems with and without observational equivalence reduction.

Problem	Synthesis Time (µs) Without Observational Equivalence Reduction	Synthesis Time (µs) With Observational Equivalence Reduction
1	47439	11745
2	11421	6040
3	592128	46845
4	1179776	83811
5	7507	4082
6	2898	2087
7	599882	48557
8	1725702	43172
9	4330286	256931

Figure 5: A table of synthesis time for nine problems with and without observational equivalence reduction.

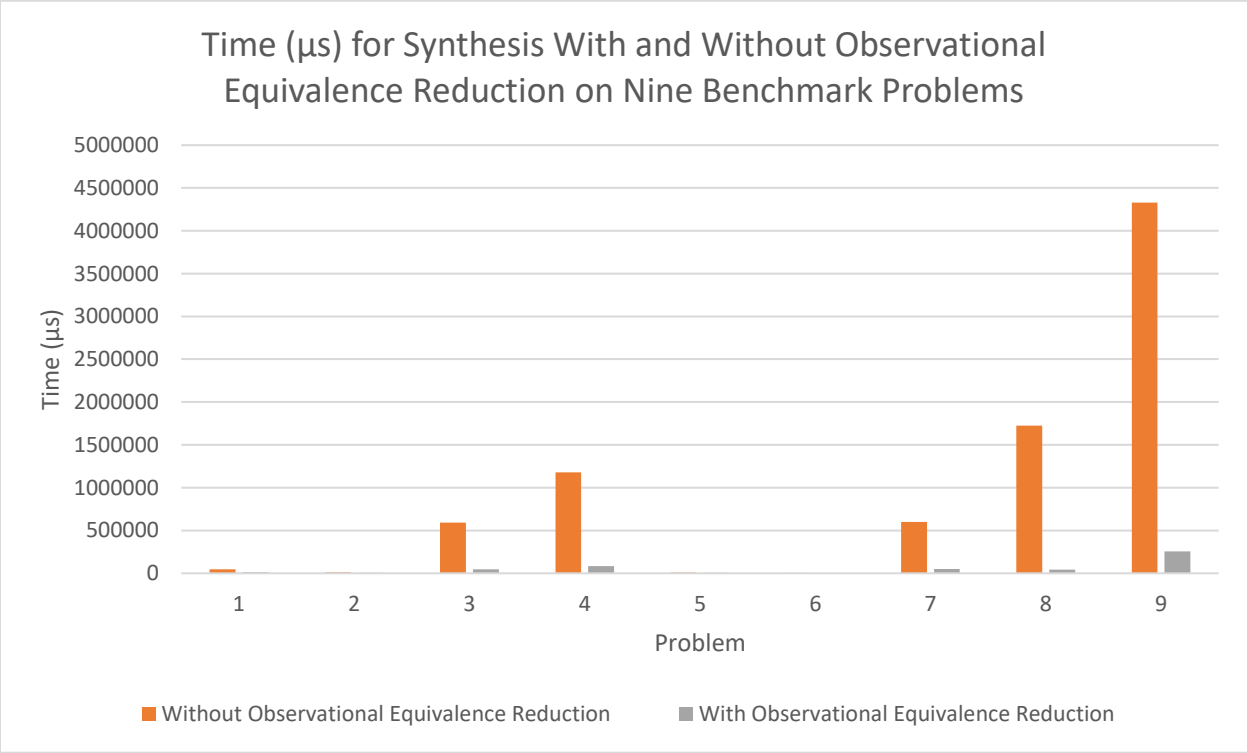

Figure 6: A graph displaying the synthesis time for nine problems with and without observational equivalence reduction.

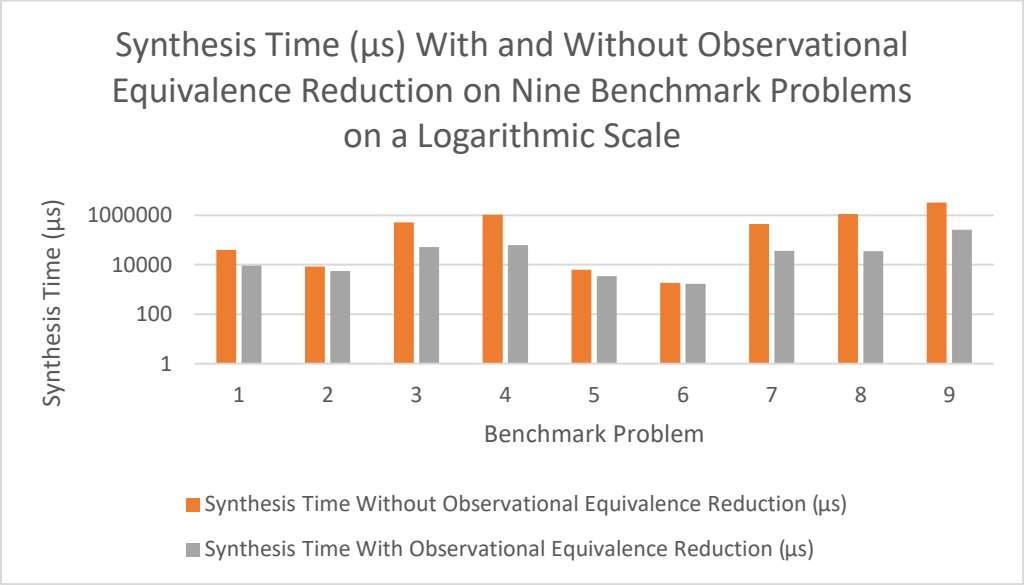

Figure 7: A graph displaying the synthesis time for nine problems with and without observational equivalence reduction on a logarithmic scale.

On all benchmarks, the synthesizer completes the problem faster with observational equivalence reduction than without. The synthesis time of the problems have a large range of 3228994 microseconds, which is expected because each problem varies wildly in difficulty. Problem 6 is an outlier with a percentage difference of only 6.86%. This can be attributed to a solution being found with low number of iterations where there is less room for optimization from observational equivalence reduction. Through the clear efficiency gains, the data suggests that observational equivalence reduction dramatically reduces the synthesis time of enumerative bottom-up program synthesis.

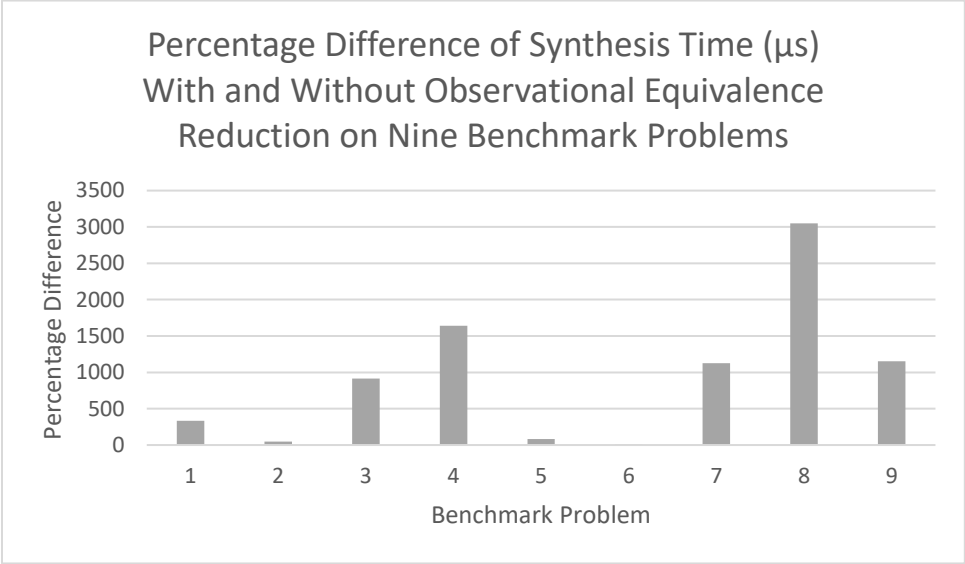

Figure 8: A graph displaying the percentage difference of synthesis time with and without observational equivalence reduction.

Higher percentage differences in speed correspond with problems that took more time, which suggests that observational equivalence reduction becomes more effective for more difficult problems. This is expected because reductions in lower height banks will exponentially decrease the size of higher banks.

Works Cited

Alur, R., Bodik, R., Juniwal, G., Martin, M. M., Raghothaman, M., Seshia, S. A., . . . Udupa, A. (2013). Syntax-guided synthesis. *2013 Formal Methods in Computer-Aided Design*. doi:10.1109/fmcad.2013.6679385

Alur, R., Singh, R., Fisman, D., & Solar-Lezama, A. (2018). Search-based program synthesis. *Communications of the ACM, 61*(12), 84-93. doi:10.1145/3208071

Barke, S., Peleg, H., & Polikarpova, N. (2020). Just-in-time learning for bottom-up enumerative synthesis. *Proceedings of the ACM on Programming Languages, 4*(OOPSLA), 1-29. doi:10.1145/3428295

Smith, C., & Albarghouthi, A. (2019). Program synthesis with equivalence reduction. *Lecture Notes in Computer Science,* 24-47. doi:10.1007/978-3-030-11245-5_2

Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages, 5*(POPL), 1-29. doi:10.1145/3434304